



E4 Educator v2.0

Tier 3 · Project 5 of 5

# P05

## Touch UI Controller

*Walark Electronics · Fundrum Engineering · May 2026*

### ★ The Final Project ★

P05 is the capstone of the entire E4 Educator curriculum. It brings every UI skill together: multi-screen touch navigation, animated transitions, modal dialogs, a number-entry keypad, MQTT-driven remote control, OTA updates, and a complete web management interface. A learner who builds this has the skills to create any embedded touch application.

### Project overview

The Touch UI Controller is a general-purpose networked control panel. It hosts a five-screen touch interface: Dashboard (live sensor readings), Controls (MQTT output buttons and sliders), Settings (NVS settings editor), Network (WiFi/MQTT status and configuration), and About (firmware info and OTA trigger). It receives MQTT messages from other E4 devices and Node-RED, and publishes button events and slider values. It is intentionally a platform — the control functions are customisable via LittleFS config files.

#### New UI concepts:

Page manager with animated slide transitions · Tab bar navigation · Modal dialog overlay · On-screen numeric keypad · Toggle buttons with state · Slider control · Scrollable list widget · MQTT-driven display updates · JSON-configurable control layout

# 1 Project specification

## 1.1 Feature specification

| ID  | Feature              | Detail                                                                                                        |
|-----|----------------------|---------------------------------------------------------------------------------------------------------------|
| F01 | Five-screen UI       | Dashboard · Controls · Settings · Network · About. Tab bar at top for instant navigation.                     |
| F02 | Touch tab bar        | Five tab buttons across the full 240px header. Active tab highlighted. Tap to switch instantly.               |
| F03 | Animated transitions | Slide-left or slide-right animation when changing screens. 100ms, sprite-based, no flicker.                   |
| F04 | Dashboard screen     | Subscribes to e4/sensor/# MQTT topics. Displays up to 4 sensor tiles. Live update on message.                 |
| F05 | Controls screen      | Up to 6 configurable controls: toggle buttons, momentary buttons, sliders. Publish to MQTT on interact.       |
| F06 | Settings screen      | Scrollable settings list. Tap to edit. On-screen numeric keypad for value entry. Saves to NVS.                |
| F07 | Network screen       | WiFi SSID/IP/signal, MQTT broker/status, uptime, boot count. Reconnect button.                                |
| F08 | About screen         | FW version, build date, board name, uptime. OTA update button — opens /update in browser via QR hint.         |
| F09 | Modal dialog         | Confirmation overlay for destructive actions (factory reset, delete log). YES/NO touch buttons.               |
| F10 | On-screen keypad     | 10-key numeric entry dialog triggered from Settings screen. Displays current and new value.                   |
| F11 | MQTT subscribe       | Subscribes to e4/sensor/# and e4/node/+/# for dashboard. Subscribes to e4/cmd/# for control feedback.         |
| F12 | JSON control config  | data/controls.json defines button labels, topics, and types. Edit file and reflash filesystem to reconfigure. |
| F13 | NVS settings         | All user preferences persisted. Factory reset via modal confirmation, not accidental button press.            |
| F14 | OTA updates          | ElegantOTA via /update. About screen shows IP and URL. Confirms update via MQTT e4/firmware.                  |

## 1.2 Screen inventory

| Screen       | Tab label | Primary content                                                                                 |
|--------------|-----------|-------------------------------------------------------------------------------------------------|
| SCREEN_DASH  | DASH      | 4 MQTT sensor tiles. Subscribes to e4/sensor/temperature, humidity, light, probe. Auto-updates. |
| SCREEN_CTRL  | CTRL      | 6 configurable controls from controls.json. Toggle buttons with visual state. Slider for value. |
| SCREEN_SETT  | SETT      | Scrollable settings list. Brightness, temp/humid alerts, log interval. Numeric keypad entry.    |
| SCREEN_NET   | NET       | WiFi SSID, signal strength, IP address. MQTT broker, port, connection status. Reconnect button. |
| SCREEN_ABOUT | INFO      | FW version, build date, uptime, boot count. OTA URL. Factory reset (modal confirm).             |



## 2 UI architecture

### 2.1 Page manager

The page manager owns screen transitions. When the active screen changes, it runs an animated slide before making the new screen active. All screens implement the same interface: `draw()`, `update()`, `handleTouch()`.

```
// PageManager.h — E4 P05 Touch UI Controller
#pragma once
#include <Arduino.h>
#include <TFT_eSPI.h>

extern TFT_eSPI tft;
extern TFT_eSprite spr;

#define SCREEN_W 240
#define SCREEN_H 280 // Content area below tab bar
#define TAB_H 40
#define ANIM_MS 100 // Transition duration
#define ANIM_STEPS 8 // Frames in animation

// All screens implement this interface
class Screen {
public:
    virtual void draw() = 0; // Full redraw
    virtual void update() = 0; // Partial update (called every loop)
    virtual void handleTouch(uint16_t tx, uint16_t ty) = 0;
    virtual const char* tabLabel() = 0;
    virtual ~Screen() {}
};

class PageManager {
public:
    Screen* screens[5];
    int activeIdx = 0;
    int numScreens = 0;

    void addScreen(Screen* s) {
        if (numScreens < 5) screens[numScreens++] = s;
    }

    void navigateTo(int idx) {
        if (idx == activeIdx || idx < 0 || idx >= numScreens) return;
        bool slideLeft = (idx > activeIdx);
        activeIdx = idx;
        animateTransition(slideLeft);
        drawTabBar();
        screens[activeIdx]->draw();
    }

    void drawTabBar() {
        tft.fillRect(0, 0, SCREEN_W, TAB_H, 0x0319);
        int tw = SCREEN_W / numScreens;
```

```

    for (int i = 0; i < numScreens; i++) {
        bool active = (i == activeIdx);
        uint16_t bg = active ? 0xFFFF : 0x0319;
        uint16_t fg = active ? 0x0319 : 0xC618;
        tft.fillRect(i*tw, 0, tw-1, TAB_H, bg);
        tft.setTextColor(fg, bg);
        tft.setTextSize(1);
        int lw = strlen(screens[i]->tabLabel()) * 6;
        tft.setCursor(i*tw + (tw-lw)/2, 15);
        tft.print(screens[i]->tabLabel());
    }
}

void handleTouch(uint16_t tx, uint16_t ty, bool touched) {
    if (!touched) return;
    if (ty < TAB_H) {
        // Tab bar tap
        int tw = SCREEN_W / numScreens;
        int idx = tx / tw;
        navigateTo(idx);
    } else {
        // Pass to active screen (offset Y by tab bar)
        screens[activeIdx]->handleTouch(tx, ty - TAB_H);
    }
}

void update() {
    screens[activeIdx]->update();
}

private:
void animateTransition(bool slideLeft) {
    // Capture current content into a snapshot
    // Slide it off-screen while new screen slides in
    // Uses two sprites: outgoing and incoming
    // Simplified version — just clear content area
    tft.fillRect(0, TAB_H, SCREEN_W, SCREEN_H, 0x0000);
    delay(ANIM_MS / 2); // Brief black flash = minimal animation
    // Full sprite-based slide: see Section 2.2
}
};

PageManager pageMgr;

```

### ★ Key point

The Screen interface enforces a consistent structure across all five screens. `draw()` does a full redraw from scratch (called on navigation). `update()` handles incremental updates each loop pass — for example, updating a sensor value without redrawing the entire screen. `handleTouch()` receives coordinates relative to the content area (tab bar Y already subtracted).

## 2.2 Sprite-based slide transition

A full sprite-based slide animation captures both screens as sprites and scrolls them horizontally in `ANIM_STEPS` frames:

```
// Full slide animation (replace animateTransition stub above)
void animateTransition(bool slideLeft) {
    // Create two sprites: outgoing and incoming
    TFT_eSprite outSprite = TFT_eSprite(&tft);
    TFT_eSprite inSprite = TFT_eSprite(&tft);
    outSprite.setColorDepth(16); outSprite.createSprite(SCREEN_W, SCREEN_H);
    inSprite.setColorDepth(16); inSprite.createSprite(SCREEN_W, SCREEN_H);

    // Render outgoing (current) screen into outSprite
    // Render incoming (new) screen into inSprite
    // (Simplified: both just fill for demo -
    // production: redirect screen draw calls to sprite)
    outSprite.fillRect(0x0319);
    inSprite.fillRect(0x0000);
    screens[activeIdx]->draw(); // Already drawn to TFT

    int stepPx = SCREEN_W / ANIM_STEPS;
    for (int step = 1; step <= ANIM_STEPS; step++) {
        int offset = step * stepPx;
        int outX = slideLeft ? -offset : offset;
        int inX = slideLeft ? SCREEN_W - offset : -SCREEN_W + offset;
        outSprite.pushSprite(outX, TAB_H);
        inSprite.pushSprite(inX, TAB_H);
        delay(ANIM_MS / ANIM_STEPS);
    }
    outSprite.deleteSprite();
    inSprite.deleteSprite();
}
```

### 3 Dashboard screen — MQTT sensor tiles

The Dashboard screen shows up to four sensor tiles, each subscribed to an MQTT topic. When the master MQTT broker receives a new value on the topic, the corresponding tile updates immediately without redrawing the whole screen.

```
// DashScreen.h
#pragma once
#include "PageManager.h"

struct SensorTile {
    const char* label;
    const char* topic;
    const char* unit;
    char        value[16];
    uint16_t    colour;
    bool        updated;
};

class DashScreen : public Screen {
public:
    SensorTile tiles[4] = {
        { "AIR TEMP",  "e4/sensor/temperature", "\u00B0C", "--", 0x07E0, false },
        { "HUMIDITY",  "e4/sensor/humidity",    "%",    "--", 0x2E97, false },
        { "LIGHT",     "e4/sensor/light",        "%",    "--", 0xFD20, false },
        { "PROBE",     "e4/sensor/probe",        "\u00B0C", "--", 0x79CF, false },
    };

    const char* tabLabel() override { return "DASH"; }

    void onMqttMessage(const char* topic, const char* value) {
        for (int i = 0; i < 4; i++) {
            if (strcmp(topic, tiles[i].topic) == 0) {
                strncpy(tiles[i].value, value, 15);
                tiles[i].updated = true;
            }
        }
    }

    void draw() override {
        tft.fillRect(0, TAB_H, SCREEN_W, SCREEN_H, 0x0000);
        for (int i = 0; i < 4; i++) drawTile(i, true);
    }

    void update() override {
        for (int i = 0; i < 4; i++) {
            if (tiles[i].updated) {
                tiles[i].updated = false;
                drawTile(i, false); // Partial update
            }
        }
    }

    void handleTouch(uint16_t tx, uint16_t ty) override {}
}
```

```
// Dashboard is read-only – no touch actions

private:
// 2x2 tile grid
int tileX(int i) { return (i % 2) * 120 + 4; }
int tileY(int i) { return (i / 2) * 130 + TAB_H + 4; }

void drawTile(int idx, bool fullDraw) {
    SensorTile& t = tiles[idx];
    int x = tileX(idx); int y = tileY(idx);
    int w = 112; int h = 122;

    if (fullDraw) {
        tft.fillRoundRect(x, y, w, h, 8, 0x162040);
        tft.drawRoundRect(x, y, w, h, 8, 0x30363D);
        tft.setTextColor(0x8B94AA, 0x162040);
        tft.setTextSize(1);
        tft.setCursor(x+8, y+8);
        tft.print(t.label);
    }

    // Value area – always update
    tft.fillRect(x+4, y+24, w-8, 60, 0x162040);
    tft.setTextColor(t.colour, 0x162040);
    tft.setTextSize(3);
    tft.setCursor(x+8, y+30);
    tft.print(t.value);
    tft.setTextSize(2);
    tft.setCursor(x+8, y+65);
    tft.setTextColor(0x8B94AA, 0x162040);
    tft.print(t.unit);
}

};

DashScreen dashScreen;
```

## 4 Controls screen — JSON-configurable buttons and sliders

The Controls screen reads its layout from data/controls.json on LittleFS. This allows the control panel to be reconfigured without reflashing firmware — only the filesystem image needs updating.

### 4.1 data/controls.json format

```
{
  "controls": [
    {
      "type": "toggle",
      "label": "LED RED",
      "topic": "e4/cmd/led/red",
      "onValue": "1",
      "offValue": "0",
      "colour": "0xF800"
    },
    {
      "type": "toggle",
      "label": "LED GREEN",
      "topic": "e4/cmd/led/green",
      "onValue": "1",
      "offValue": "0",
      "colour": "0x07E0"
    },
    {
      "type": "momentary",
      "label": "ALERT",
      "topic": "e4/cmd/alert",
      "value": "on",
      "colour": "0xFD20"
    },
    {
      "type": "momentary",
      "label": "SILENCE",
      "topic": "e4/cmd/alert",
      "value": "off",
      "colour": "0xC618"
    },
    {
      "type": "slider",
      "label": "BRIGHTNESS",
      "topic": "e4/cmd/brightness",
      "minVal": 0,
      "maxVal": 255,
      "colour": "0x2E97"
    },
    {
      "type": "slider",
      "label": "VOLUME",
      "topic": "e4/cmd/volume",
      "minVal": 0,
      "maxVal": 100,
      "colour": "0xFD20"
    }
  ]
}
```

```

    }
  ]
}

```

## 4.2 ControlScreen implementation

```

// CtrlScreen.h (abbreviated)
#pragma once
#include "PageManager.h"
#include <ArduinoJson.h>
#include <LittleFS.h>

enum CtrlType { CTRL_TOGGLE, CTRL_MOMENTARY, CTRL_SLIDER };

struct Control {
  CtrlType type;
  char    label[16];
  char    topic[48];
  char    onVal[8];
  char    offVal[8];
  uint16_t colour;
  int     minVal, maxVal;
  bool    state;          // Toggle on/off
  int     sliderVal;      // Slider position
};

#define MAX_CONTROLS 6

class CtrlScreen : public Screen {
public:
  Control controls[MAX_CONTROLS];
  int     numControls = 0;

  const char* tabLabel() override { return "CTRL"; }

  bool loadFromJSON() {
    File f = LittleFS.open("/controls.json", "r");
    if (!f) { Serial.println("controls.json not found."); return false; }
    StaticJsonDocument<1024> doc;
    if (deserializeJson(doc, f) != DeserializationError::Ok) {
      f.close(); return false;
    }
    f.close();

    JsonArray arr = doc["controls"];
    numControls = 0;
    for (JsonObject obj : arr) {
      if (numControls >= MAX_CONTROLS) break;
      Control& c = controls[numControls];
      String type = obj["type"] | "toggle";
      c.type = (type == "slider") ? CTRL_SLIDER :
               (type == "momentary") ? CTRL_MOMENTARY : CTRL_TOGGLE;
      strncpy(c.label, obj["label"] | "BTN", 15);
    }
  }
};

```

```

        strncpy(c.topic, obj["topic"] | "e4/cmd", 47);
        strncpy(c.onVal, obj["onValue"] | "1", 7);
        strncpy(c.offVal, obj["offValue"] | "0", 7);
        c.colour = (uint16_t) strtoul(obj["colour"] | "0xFFFF", nullptr, 16);
        c.minVal = obj["minVal"] | 0;
        c.maxVal = obj["maxVal"] | 100;
        c.state = false;
        c.sliderVal = c.minVal;
        numControls++;
    }
    Serial.printf("Loaded %d controls.\n", numControls);
    return true;
}

void draw() override {
    tft.fillRect(0, TAB_H, SCREEN_W, SCREEN_H, 0x0000);
    for (int i = 0; i < numControls; i++) drawControl(i);
}

void handleTouch(uint16_t tx, uint16_t ty) override;
// defined below

void update() override {}

private:
    // 2x3 grid: 2 columns, 3 rows, 80px tall each
    int ctrlX(int i) { return (i % 2) * 120 + 4; }
    int ctrlY(int i) { return TAB_H + (i / 2) * 88 + 4; }

    void drawControl(int idx) {
        Control& c = controls[idx];
        int x = ctrlX(idx); int y = ctrlY(idx);
        int w = 112; int h = 80;

        if (c.type == CTRL_SLIDER) {
            // Slider: label + horizontal track + thumb
            tft.fillRoundRect(x, y, w, h, 6, 0x162040);
            tft.setTextColor(0xC618, 0x162040);
            tft.setTextSize(1); tft.setCursor(x+4, y+4);
            tft.print(c.label);
            tft.setTextSize(1); tft.setCursor(x+4, y+16);
            tft.printf("%d", c.sliderVal);
            // Track
            int trackX = x+8; int trackY = y+40;
            int trackW = w-16;
            tft.drawRect(trackX, trackY, trackW, 10, 0x30363D);
            int thumbX = trackX + map(c.sliderVal,
                                    c.minVal, c.maxVal, 0, trackW-8);
            tft.fillRect(thumbX, trackY, 8, 10, c.colour);
        } else {
            // Button: filled when state ON
            bool on = (c.type == CTRL_TOGGLE) && c.state;
            uint16_t bg = on ? c.colour : 0x162040;
            uint16_t fg = on ? 0x000000 : c.colour;

```

```

        tft.fillRoundRect(x, y, w, h, 8, bg);
        tft.drawRoundRect(x, y, w, h, 8, c.colour);
        tft.setTextColor(fg, bg);
        tft.setTextSize(2);
        int lw = strlen(c.label) * 12;
        tft.setCursor(x + (w-lw)/2, y + (h-16)/2);
        tft.print(c.label);
    }
}
};

void CtrlScreen::handleTouch(uint16_t tx, uint16_t ty) {
    for (int i = 0; i < numControls; i++) {
        Control& c = controls[i];
        int x = ctrlX(i) - 4; int y = ctrlY(i) - TAB_H;
        if (tx < x || tx > x+120 || ty < y || ty > y+88) continue;

        if (c.type == CTRL_TOGGLE) {
            c.state = !c.state;
            mqtt.publish(c.topic, c.state ? c.onVal : c.offVal, false);
            drawControl(i);
        } else if (c.type == CTRL_MOMENTARY) {
            mqtt.publish(c.topic, c.onVal, false);
            // Flash feedback
            c.state = true; drawControl(i);
            delay(120);
            c.state = false; drawControl(i);
        } else if (c.type == CTRL_SLIDER) {
            // Map touch X to slider value
            int trackX = ctrlX(i) + 8;
            int trackW = 112 - 16;
            int newVal = map(constrain(tx - trackX, 0, trackW),
                           0, trackW, c.minVal, c.maxVal);
            c.sliderVal = newVal;
            char buf[8]; sprintf(buf, "%d", newVal);
            mqtt.publish(c.topic, buf, false);
            drawControl(i);
        }
    }
}

CtrlScreen ctrlScreen;

```

## 5 On-screen numeric keypad

The numeric keypad is a modal overlay drawn on top of the current screen. It is triggered from the Settings screen when a value tile is tapped, and returns the entered value to a callback function.

```
// NumericKeypad.h — modal numeric entry overlay
#pragma once
#include <Arduino.h>
#include <TFT_eSPI.h>

extern TFT_eSPI tft;

class NumericKeypad {
public:
    typedef void (*Callback)(int value);

    bool    active    = false;
    int     current   = 0;    // Existing value (shown as hint)
    int     entered   = 0;    // Value being entered
    int     digits    = 0;
    Callback callback = nullptr;

    void show(int existingVal, Callback cb) {
        current = existingVal;
        entered = 0; digits = 0;
        callback = cb;
        active = true;
        drawKeypad();
    }

    void handleTouch(uint16_t tx, uint16_t ty) {
        if (!active) return;
        // Overlay starts at Y=60, keys in 4x3 grid
        if (ty < 60 || ty > 310) return;
        int row = (ty - 100) / 52;
        int col = tx / 80;
        if (row < 0 || row > 3 || col < 0 || col > 2) return;

        int keys[4][3] = {{1,2,3},{4,5,6},{7,8,9},{-1,0,-2}};
        // -1 = CLR, -2 = OK
        int key = keys[row][col];

        if (key == -1) { // CLR
            entered = 0; digits = 0;
        } else if (key == -2) { // OK
            if (callback) callback(entered);
            active = false;
            tft.fillRect(0, 60, 240, 260, 0x0000);
            return;
        } else if (digits < 5) {
            entered = entered * 10 + key;
            digits++;
        }
        drawKeypad();
    }
};
```

```

    }

private:
    void drawKeypad() {
        // Semi-transparent overlay background
        tft.fillRect(10, 60, 220, 260, 10, 0x162040);
        tft.drawRoundRect(10, 60, 220, 260, 10, 0x2E86AB);

        // Title and current value
        tft.setTextColor(0xC618, 0x162040);
        tft.setTextSize(1); tft.setCursor(20, 72);
        tft.printf("Current: %d", current);

        // New value display
        tft.setTextColor(0xFFFF, 0x162040);
        tft.setTextSize(3); tft.setCursor(20, 90);
        tft.printf("> %d", entered);

        // Key grid
        const char* labels[] = {"1","2","3","4","5","6","7","8","9","CLR","0","OK"};
        uint16_t btnCols[] = {
            0x18C3,0x18C3,0x18C3,0x18C3,0x18C3,0x18C3,
            0x18C3,0x18C3,0x18C3,0xF800,0x18C3,0x07E0
        };
        for (int i = 0; i < 12; i++) {
            int x = 14 + (i % 3) * 74;
            int y = 130 + (i / 3) * 48;
            tft.fillRect(x, y, 64, 40, 6, btnCols[i]);
            tft.setTextColor(0xFFFF, btnCols[i]);
            tft.setTextSize(2);
            int lw = strlen(labels[i]) * 12;
            tft.setCursor(x + (64-lw)/2, y+12);
            tft.print(labels[i]);
        }
    }
};

NumericKeypad numKeypad;

```

## 6 Settings, Network, and About screens

### 6.1 Settings screen

The Settings screen shows a scrollable list of NVS settings. Tapping a row opens the numeric keypad. On confirmation, the new value is saved to NVS and applied immediately.

```
// SettScreen.h (abbreviated)
#pragma once
#include "PageManager.h"
#include "NumericKeypad.h"
#include "Settings.h"

extern SettingsManager settings;
extern NumericKeypad numKeypad;

class SettScreen : public Screen {
public:
    const char* tabLabel() override { return "SETT"; }

    struct SettItem { const char* label; const char* unit; int value; };

    SettItem items[5] = {
        { "Brightness",    "/255", 200 },
        { "Temp alert",    "\u00B0C", 28  },
        { "Humid alert",   "%",    70  },
        { "Log interval",  "s",     5   },
        { "Gain",          "x",     1   },
    };

    void draw() override {
        tft.fillRect(0, TAB_H, SCREEN_W, SCREEN_H, 0x0000);
        loadValues();
        for (int i = 0; i < 5; i++) drawItem(i, false);
    }

    void handleTouch(uint16_t tx, uint16_t ty) override {
        int idx = ty / 52;
        if (idx < 0 || idx >= 5) return;
        int existing = items[idx].value;
        numKeypad.show(existing, [](int newVal) {
            // Apply: update settings + save
            // (Full implementation uses captured idx)
            settings.save();
        });
    }

    void update() override {
        if (numKeypad.active)
            numKeypad.handleTouch(0, 0); // handled in main loop
    }

private:
```

```

void loadValues() {
    items[0].value = settings.s.brightness;
    items[1].value = (int)settings.s.tempAlert;
    items[2].value = (int)settings.s.humidAlert;
    items[3].value = (int)(settings.s.logInterval / 1000);
}

void drawItem(int idx, bool highlight) {
    int y = TAB_H + idx * 52;
    uint16_t bg = highlight ? 0x0A29 : 0x0D1117;
    tft.fillRect(0, y, SCREEN_W, 50, bg);
    tft.drawFastHLine(0, y+50, SCREEN_W, 0x30363D);
    tft.setTextColor(0xC618, bg);
    tft.setTextSize(1); tft.setCursor(8, y+8);
    tft.print(items[idx].label);
    tft.setTextColor(0xFFFF, bg);
    tft.setTextSize(2); tft.setCursor(8, y+24);
    tft.printf("%d %s", items[idx].value, items[idx].unit);
}
};

```

## 6.2 Network screen

```

// NetScreen.h (abbreviated)
class NetScreen : public Screen {
public:
    const char* tabLabel() override { return "NET"; }

    void draw() override {
        tft.fillRect(0, TAB_H, SCREEN_W, SCREEN_H, 0x0000);
        refresh();
    }

    void update() override {
        static unsigned long lastRefresh = 0;
        if (millis() - lastRefresh > 2000) {
            lastRefresh = millis(); refresh();
        }
    }

    void handleTouch(uint16_t tx, uint16_t ty) override {
        // RECONNECT button at Y 220
        if (ty > 200 && ty < 250) {
            WiFi.reconnect();
            Serial.println("Manual reconnect triggered.");
        }
    }

private:
    void refresh() {
        int y = TAB_H + 8;
        tft.setTextSize(1);
    }
};

```

```

    auto row = [&](const char* lbl, const String& val, uint16_t col) {
        tft.setTextColor(0x8B94AA, 0x0000);
        tft.setCursor(8, y); tft.print(lbl);
        tft.setTextColor(col, 0x0000);
        tft.setCursor(100, y); tft.print(val);
        y += 20;
    };

    bool wok = WiFi.status() == WL_CONNECTED;
    bool mok = mqtt.connected();
    row("WiFi SSID:", WiFi.SSID(), wok ? 0x07E0 : 0xF800);
    row("IP:", wok ? WiFi.localIP().toString() : "---",
        0xFFFF);
    row("Signal:", wok ? String(WiFi.RSSI())+" dBm" : "---",
        0x79CF);

    y += 8;
    row("MQTT broker:", mqttBroker, mok ? 0x07E0 : 0xFFE0);
    row("MQTT status:", mok ? "Connected" : "Disconnected",
        mok ? 0x07E0 : 0xF800);

    y += 8;
    row("Uptime:", String(millis()/1000) + "s", 0xC618);
    row("Boot #:",
        String(settings.s.bootCount), 0xC618);

    // Reconnect button
    tft.fillRect(8, y+20, 100, 36, 6, 0x0319);
    tft.drawRoundRect(8, y+20, 100, 36, 6, 0x2E86AB);
    tft.setTextColor(0xFFFF, 0x0319);
    tft.setTextSize(1); tft.setCursor(22, y+31);
    tft.print("RECONNECT");
}
};

```

### 6.3 About screen with modal factory reset

```

// AboutScreen.h (abbreviated)
class AboutScreen : public Screen {
public:
    const char* tabLabel() override { return "INFO"; }
    bool showResetModal = false;

    void draw() override {
        tft.fillRect(0, TAB_H, SCREEN_W, SCREEN_H, 0x0000);
        int y = TAB_H + 12;
        tft.setTextColor(0xC618, 0x0000); tft.setTextSize(1);

        auto line = [&](const char* l, const String& v) {
            tft.setCursor(8, y); tft.print(l);
            tft.setTextColor(0xFFFF, 0x0000);
            tft.setCursor(110, y); tft.print(v); y += 22;
            tft.setTextColor(0xC618, 0x0000);
        };
    }
};

```

```

    line("Firmware:", FW_VERSION);
    line("Build date:", FW_DATE);
    line("Board:", "E4 Educator v2.0");
    line("Uptime:", String(millis()/1000) + "s");

    tft.setTextColor(0x2E97, 0x0000);
    tft.setCursor(8, y); y += 22;
    if (WiFi.status() == WL_CONNECTED)
        tft.printf("OTA: http://%s/update",
                    WiFi.localIP().toString().c_str());
    else
        tft.print("OTA: connect WiFi first");

    // Factory reset button
    tft.fillRect(8, y+10, 140, 36, 6, 0x300A0A);
    tft.drawRoundRect(8, y+10, 140, 36, 6, 0xF800);
    tft.setTextColor(0xF800, 0x300A0A);
    tft.setTextSize(1); tft.setCursor(22, y+21);
    tft.print("FACTORY RESET");
}

void handleTouch(uint16_t tx, uint16_t ty) override {
    if (showResetModal) {
        // YES button
        if (tx > 20 && tx < 100 && ty > 180 && ty < 220) {
            settings.reset(); ESP.restart();
        }
        // NO button
        if (tx > 140 && tx < 220 && ty > 180 && ty < 220) {
            showResetModal = false;
            draw();
        }
        return;
    }
    // Factory reset button area (approx)
    if (ty > 180 && ty < 220 && tx < 150) {
        showResetModal = true;
        drawModal();
    }
}

void update() override {}

private:
void drawModal() {
    tft.fillRect(20, TAB_H+60, 200, 160, 10, 0x162040);
    tft.drawRoundRect(20, TAB_H+60, 200, 160, 10, 0xF800);
    tft.setTextColor(0xFFFF, 0x162040);
    tft.setTextSize(2); tft.setCursor(35, TAB_H+80);
    tft.print("Factory reset?");
    tft.setTextSize(1); tft.setCursor(30, TAB_H+110);
    tft.print("All settings will be");
    tft.setCursor(30, TAB_H+125);
    tft.print("cleared. Reboot follows.");
}

```

```
// YES
tft.fillRect(30, TAB_H+150, 70, 36, 6, 0xF800);
tft.setTextColor(0xFFFF, 0xF800);
tft.setTextSize(2); tft.setCursor(44, TAB_H+161);
tft.print("YES");
// NO
tft.fillRect(140, TAB_H+150, 70, 36, 6, 0x18C3);
tft.setTextColor(0xFFFF, 0x18C3);
tft.setCursor(158, TAB_H+161);
tft.print("NO");
}
};
```

## 7 main.cpp — wiring it all together

```
#include <Arduino.h>
#include <esp_ota_ops.h>
#include "PageManager.h"
#include "DashScreen.h"
#include "CtrlScreen.h"
#include "SettScreen.h"
#include "NetScreen.h"
#include "AboutScreen.h"
#include "NumericKeypad.h"
#include "Network.h"
#include "Settings.h"
#include "Button.h"

TFT_eSPI    tft;
TFT_eSprite spr = TFT_eSprite(&tft);
Button      nextBtn(BTN_NEXT);
Button      entBtn(BTN_ENT);
SettingsManager settings;

DashScreen  dashScreen;
CtrlScreen  ctrlScreen;
SettScreen  settScreen;
NetScreen   netScreen;
AboutScreen aboutScreen;

bool otaValid    = false;
bool webStarted  = false;

// MQTT message handler — route to dashboard
void mqttCallback(char* topic, byte* payload, unsigned int len) {
    char val[32]; val[min((int)len, 31)] = 0;
    memcpy(val, payload, min((int)len, 31));
    dashScreen.onMqttMessage(topic, val);
    // Also handle e4/cmd/brightness etc. for self-control
}

void setup() {
    Serial.begin(115200);
    Serial.printf("P05 Touch UI  FW:%s\n", FW_VERSION);

    settings.load();

    // LEDs
    pinMode(LED_GREEN, OUTPUT); digitalWrite(LED_GREEN, LOW);
    pinMode(LED_RED,   OUTPUT); digitalWrite(LED_RED,   LOW);
    pinMode(LED_BLUE,  OUTPUT); digitalWrite(LED_BLUE,  LOW);

    // TFT
    pinMode(TFT_BL, OUTPUT); digitalWrite(TFT_BL, LOW);
    tft.init(); tft.setRotation(0); tft.fillScreen(0x0000); // Landscape 320x240
    ledcSetup(0, 5000, 8); ledcAttachPin(TFT_BL, 0);
```

```
    ledcWrite(0, settings.s.brightness);

    spr.setColorDepth(16); spr.createSprite(320, 165);

    nextBtn.begin(); entBtn.begin();

    // Touch calibration
    Preferences p; p.begin("touch", true);
    uint16_t cal[5];
    if (p.getBytesLength("cal") == sizeof(cal)) {
        p.getBytes("cal", cal, sizeof(cal)); tft.setTouch(cal);
    }
    p.end();

    // Load control config from LittleFS
    if (LittleFS.begin(true)) {
        ctrlScreen.loadFromJSON();
    }

    // Register screens
    pageMgr.addScreen(&dashScreen);
    pageMgr.addScreen(&ctrlScreen);
    pageMgr.addScreen(&settScreen);
    pageMgr.addScreen(&netScreen);
    pageMgr.addScreen(&aboutScreen);

    // Network
    initNetwork();
    mqtt.setCallback(mqttCallback);

    // Draw initial UI
    pageMgr.drawTabBar();
    pageMgr.screens[0]->draw();

    digitalWrite(LED_GREEN, HIGH);
    Serial.println("P05 Touch UI ready.");
}

void loop() {
    unsigned long now = millis();

    wifiUpdate(now); mqttUpdate(now); ElegantOTA.loop();

    if (wifiReady() && !webStarted) {
        setupWebServer();
        mqtt.subscribe("e4/sensor/#");
        mqtt.subscribe("e4/node/+/temperature");
        webStarted = true;
    }

    Button::Event nextEv = nextBtn.read();
    Button::Event entEv = entBtn.read();
}
```

```
// NEXT cycles screens
if (nextEv == Button::SHORT_PRESS)
    pageMgr.navigateTo((pageMgr.activeIdx + 1) % pageMgr.numScreens);

// Touch handling
uint16_t tx, ty;
bool touched = tft.getTouch(&tx, &ty);
if (touched) {
    if (numKeypad.active) {
        numKeypad.handleTouch(tx, ty);
    } else {
        static unsigned long lastTap = 0;
        if (now - lastTap > 300) {
            lastTap = now;
            pageMgr.handleTouch(tx, ty, touched);
        }
    }
}

pageMgr.update();

if (!otaValid && now > 10000) {
    esp_ota_mark_app_valid_cancel_rollback();
    otaValid = true;
}
}
```

## 8 Commissioning checklist

| <input type="checkbox"/> | Step                                                                                               |
|--------------------------|----------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | Upload firmware (Ctrl+Alt+U). Upload filesystem (Upload Filesystem Image).                         |
| <input type="checkbox"/> | On boot: confirm tab bar visible with DASH · CTRL · SETT · NET · INFO tabs.                        |
| <input type="checkbox"/> | Tap DASH: confirm four sensor tiles visible, all showing "--" (no data yet).                       |
| <input type="checkbox"/> | Tap CTRL: confirm six control buttons/sliders loaded from controls.json.                           |
| <input type="checkbox"/> | Tap SETT: confirm five settings rows with labels and current values.                               |
| <input type="checkbox"/> | Tap NET: confirm WiFi SSID, IP address visible after WiFi connects.                                |
| <input type="checkbox"/> | Tap INFO: confirm FW_VERSION, build date, and OTA URL visible.                                     |
| <input type="checkbox"/> | Tap a setting row in SETT. Confirm numeric keypad overlay appears.                                 |
| <input type="checkbox"/> | Enter a new value on keypad. Tap OK. Confirm value updates in list.                                |
| <input type="checkbox"/> | Power cycle. Confirm changed value persists in SETT screen.                                        |
| <input type="checkbox"/> | Tap a toggle button in CTRL. Confirm button highlights. Check MQTT Explorer for published message. |
| <input type="checkbox"/> | Tap ALERT momentary button. Confirm brief highlight and MQTT publish.                              |
| <input type="checkbox"/> | Drag slider in CTRL. Confirm value updates and MQTT publishes on drag.                             |
| <input type="checkbox"/> | With P01 or P02 running on another E4: confirm DASH screen updates sensor tiles via MQTT.          |
| <input type="checkbox"/> | Tap INFO → FACTORY RESET. Confirm modal dialog appears with YES/NO.                                |
| <input type="checkbox"/> | Tap NO. Confirm dialog dismisses and settings intact.                                              |
| <input type="checkbox"/> | Test animated transition: tap between tabs. Confirm brief slide animation.                         |
| <input type="checkbox"/> | Test OTA: build new firmware, navigate to OTA URL on About screen, upload.                         |

## 9 The E4 Educator curriculum — complete

Completing P05 marks the end of the E4 Educator v2.0 curriculum. The table below is a consolidated record of everything covered across all 16 tutorials and 5 projects.

| T/P | Topic               | Skills gained                                                 |
|-----|---------------------|---------------------------------------------------------------|
| T01 | PlatformIO & the E4 | Toolchain, build_flags, millis() timing, Serial debug         |
| T02 | GPIO & build_flags  | All GPIO modes, ADC1 rule, LEDC PWM, boot-sensitive pins      |
| T03 | Buttons & debounce  | Contact bounce, Button.h, short/long press state machine      |
| T04 | I2C fundamentals    | Bus protocol, scanner, AHT20, multi-device, Wire.begin()      |
| T05 | OLED display        | SSD1306, buffer model, zone layout, multi-page navigation     |
| T06 | One-wire & analog   | DS18B20, non-blocking state machine, LDR, averaging           |
| T07 | Audio output        | TonePlayer.h, DAC waveforms, PWM transistor, Tier 1 capstone  |
| T08 | TFT & sprites       | ILI9341, RGB565, sprite pipeline, zone layout, colour palette |

|     |                         |                                                                                               |
|-----|-------------------------|-----------------------------------------------------------------------------------------------|
| T09 | Touch input             | XPT2046, calibration, TouchZone.h, SPI coexistence                                            |
| T10 | SD card & file I/O      | TFT-first rule, FAT32, CSV logging, BMP splash, config files                                  |
| T11 | NVS persistence         | Preferences, 15-char key rule, Settings.h, factory reset                                      |
| T12 | WiFi & MQTT             | Non-blocking state machine, PubSubClient, LWT, Node-RED                                       |
| T13 | LittleFS                | Partition, web server, smooth fonts, ArduinoJson                                              |
| T14 | OTA updates             | ElegantOTA, progress bar, versioning, rollback                                                |
| T15 | ESP-NOW                 | MAC addresses, SensorPacket.h, star network, MQTT bridge                                      |
| T16 | I2S & MEMS mic          | ICS43434, I2S config, FFT, BANDSCOPE, event detection                                         |
| P01 | Environmental Monitor   | All Tier 1+2 skills integrated. Multi-module architecture. AHT20 + DS18B20 + LDR.             |
| P02 | Data Logger             | DS3231 RTC + NTP, BMP280 pressure trend, daily log rotation, Chart.js web graph, CSV download |
| P03 | Wireless Sensor Network | ESP-NOW star network, deep sleep nodes, battery monitor, 4-card TFT, per-node SD logging      |
| P04 | BANDSCOPE               | Dual-core FreeRTOS, four display modes, octave bands, event detection engine, MQTT telemetry  |
| P05 | Touch UI Controller     | Page manager, animated transitions, JSON-configurable controls, numeric keypad, modal dialogs |

## ★ E4 Educator v2.0 — Curriculum Complete ★

16 tutorials · 5 integrated projects · 5 reusable toolkit files · 1 reference card

A learner who has completed this curriculum can design, build, and deploy professional-grade ESP32 firmware. They understand peripheral hardware, wireless protocols, real-time display systems, data persistence, over-the-air updates, and multi-sensor network architectures.

*Walark Electronics · Fundrum Engineering · Christchurch, New Zealand · May 2026*

## 10 Extending the project

- Add a colour theme selector: store a theme index in NVS, reload the colour palette on boot. Create three themes — Dark (default), Light, and High Contrast.
- Add a chart screen: a sixth tab showing a live scrolling line graph of one MQTT topic. Use the same sprite-rendering approach as the spectrum display in P04.
- Add a keyboard screen for SSID/password entry: a full QWERTY touch keyboard for network provisioning via the touch screen, eliminating the need for a serial connection for first-time WiFi setup.
- Add a macro system: define sequences of MQTT publishes as macros in controls.json. A single button tap triggers multiple topic publishes in sequence with optional delays.
- Add a voice-activated mode using the MEMS microphone from T16: clap once to wake the display from dim, clap twice to publish a configurable MQTT event.
- Integrate P03 as a sub-panel: add a fifth tab showing the P03 wireless sensor network node grid, making P05 a unified control panel for an entire E4 installation.

*E4 Educator v2.0 · P05: Touch UI Controller · Walark Electronics · Fundrum Engineering · May 2026*